Vectors as Representations

Neural population states, feature space, length, direction and basis

WHAT YOU WILL LEARN

- What a vector is, and why one population response and one row of a dataset are the same kind of object
- How to read a vector three ways, as a list, an arrow and a point, and when each view helps
- How to add, subtract and scale vectors, and what each operation means for neural data
- How the Euclidean norm follows from Pythagoras, what distance measures, and when it misleads
- How to normalise a vector, and what normalisation keeps and throws away
- What linear combinations, span, basis, coordinates and dimension mean, and why population activity often fills only part of the space
- How a recording through time becomes a trajectory of vectors

Before you start

These notes are the mathematical companion to the Lesson 1 teaching notebook. The notebook lets you *see* each idea; these notes explain *why* it is true. Read these notes before the live session, it will make it easier for you to follow the teaching and coding session. You need nothing beyond school algebra, coordinates on a graph, and Pythagoras' theorem.

Boxes marked Why it is true hold the derivations. You can skip them on a first read, but every formula in these notes is proved somewhere, usually in a few lines.

Table 1.1 *Notation used in this lesson.*

Symbol	Meaning
$a,\ c,\ \alpha$	scalars: single numbers (plain letters)
$\mathbf{v}, \mathbf{r}, \mathbf{x}$	vectors (bold lower case)
v_i	the i-th coordinate of $\mathbf{v}$ (a single number, not bold)
n	the number of coordinates, the *dimension*
$\mathbb{R}^n$	the set of all vectors with n real-number coordinates
$\mathbf{0}$	the zero vector $(0, \ldots, 0)$
$\|\mathbf{v}\|$	the length (Euclidean norm) of $\mathbf{v}$
$d(\mathbf{u}, \mathbf{v})$	the distance between $\mathbf{u}$ and $\mathbf{v}$
$\hat{\mathbf{v}}$	the unit vector in the direction of $\mathbf{v}$
$\mathbf{e}_1, \ldots, \mathbf{e}_n$	the standard basis vectors
$\mathbf{v}^\mathsf{T}$	the transpose of $\mathbf{v}$: a column turned into a row

1.1. The problem: one brain state as one vector

▶ In the teaching notebook: Motivation

Three neurons are recorded at the same time. At one moment their firing rates are 12, 4 and 19 spikes per second. Shown a different stimulus, the same three neurons fire at 50, 8 and 33 spikes per second.

You could treat these as six unrelated numbers. But each triple describes *one* thing: the state of a small population at one instant. Writing each triple as a single object,

$$\mathbf{r}_1 = (12,\ 4,\ 19), \qquad \mathbf{r}_2 = (50,\ 8,\ 33) \quad \text{spikes/s},$$

lets you ask questions about states as wholes.

- How did the state change? Subtract: $\mathbf{r}_2 - \mathbf{r}_1 = (38,\ 4,\ 14)$.
- How far did it move? Measure the length of that change: $\sqrt{38^2 + 4^2 + 14^2} = 40.69$ spikes/s.

Both answers are explained in this lesson.

PYTHON

```
r_1 = np.array([12.0, 4.0, 19.0])   # spikes/s, (N1, N2, N3), stimulus 1
r_2 = np.array([50.0, 8.0, 33.0])   # spikes/s, stimulus 2

print(r_2 - r_1)                    # -> [38.  4. 14.]
print(np.linalg.norm(r_2 - r_1))    # -> 40.69
```

The same object appears in machine learning. One house in a dataset of house prices is described by its size, number of bedrooms and age, $\mathbf{x} = (1416,\ 3,\ 40)$. A model that predicts the price receives this whole list as its input. A population response and a row of a dataset are the same kind of mathematical object: a **vector**.

1.2. Scalars and vectors

▶ In the teaching notebook: Part 1

DEFINITION – SCALAR AND VECTOR

A **scalar** is a single number, such as one neuron's firing rate of 23.5 spikes/s. A **vector** is an *ordered* list of n numbers treated as one object,

$$\mathbf{v} = (v_1,\ v_2,\ \ldots,\ v_n).$$

The numbers v_i are its **coordinates** (or components), and n is its **dimension**. The set of all such vectors is written $\mathbb{R}^n$.

Three words in that definition carry most of the meaning.

Ordered. $(12,\ 4,\ 19)$ and $(4,\ 12,\ 19)$ are different vectors. Coordinate i always means the same thing: the first coordinate is always neuron N1, the second always N2. If two recordings list their neurons in different orders, their vectors cannot be compared until the order is matched.

One object. A vector is not a bag of numbers. Operations on it act on the whole state: adding two states, scaling a state, measuring its length.

Dimension. The dimension is the number of coordinates, nothing more. "High-dimensional" simply means "many coordinates".

Table 1.2 *Vectors you will meet, and their dimensions.*

Object	Example	Dimension
Population response of 3 neurons	$(12,\ 4,\ 19)$ spikes/s	3
One time point from a 64-electrode EEG cap	one voltage per electrode	64
One 28×28 greyscale image, flattened	one brightness per pixel	784
One fMRI volume	one signal per voxel	$\sim$100 000
One house in a dataset	(size, bedrooms, age)	3
Model parameters	$\mathbf{w} = (w_1, w_2, w_3)$	3

The last row matters later. The numbers a model *learns*, such as one weight per input, also form a vector, usually written $\mathbf{w}$ or $\boldsymbol{\theta}$. Everything in this lesson applies to them too.

KEY POINT

A vector does not carry its own meaning. The list $(12,\ 4,\ 19)$ could be three firing rates, or a house, or nothing at all. Meaning lives in two pieces of information you must keep alongside the numbers: what each coordinate refers to, and its units.

1.3. Three views of a vector

▶ In the teaching notebook: Part 2

The vector $\mathbf{v} = (4,\ -2)$ can be read in three ways:

1. **A list of numbers.** Two values stored in order.
2. **An arrow (a displacement).** Starting anywhere, move 4 units along the first axis and -2 along the second. The arrow says *how to move*, not where you are.
3. **A point (a state).** The location $(4,\ -2)$ itself. The point says *where you are*.

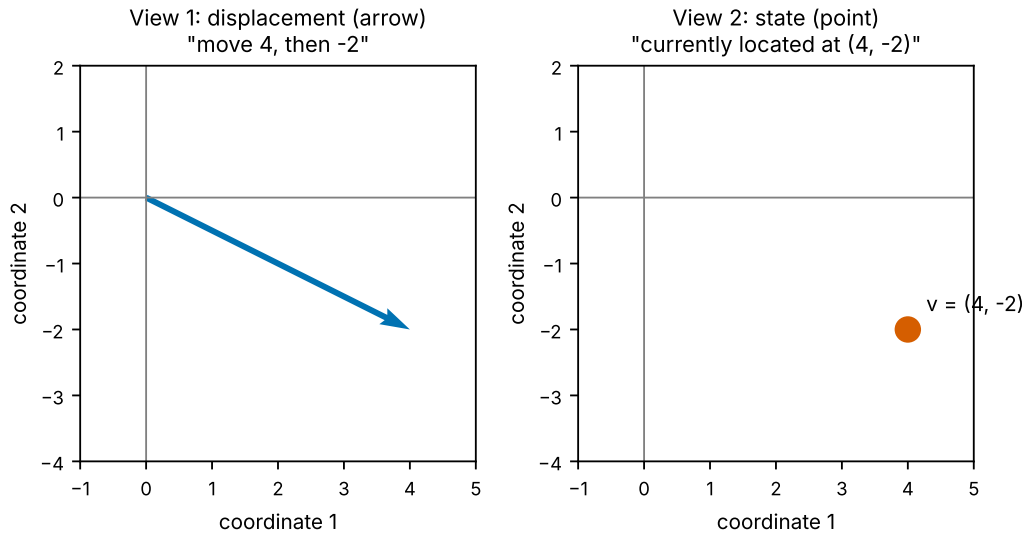


Figure 1.1 *The same vector drawn as a displacement (left) and as a state (right).*

For neural data the point view dominates. A population response is drawn as a point in **neural state space**, with one axis per neuron (Figure 1.2). Repeated trials of one condition form a cloud of nearby points; different conditions form different clouds. The arrow view takes over when we ask how one state *changes* into another.

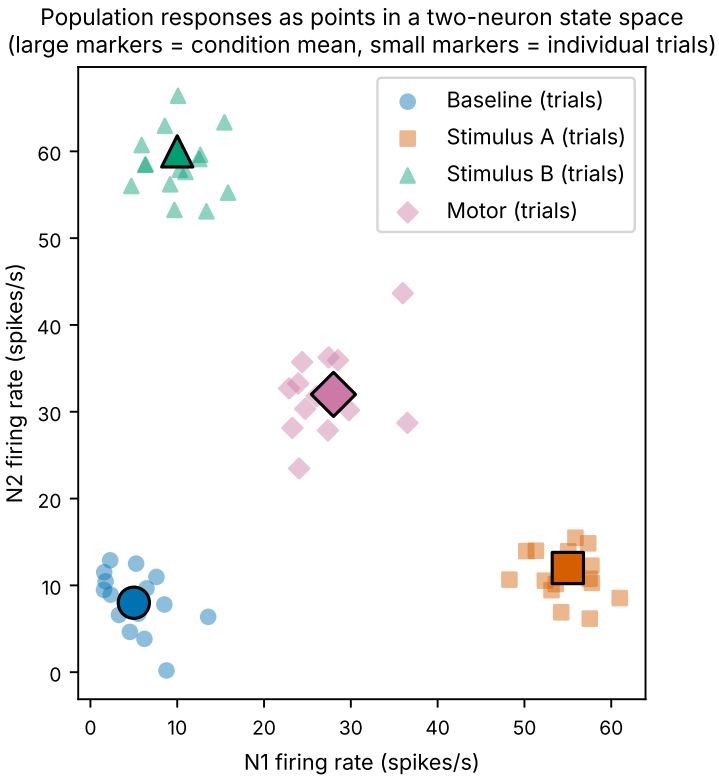


Figure 1.2 *Population responses to four conditions as points in two-neuron state space. Small markers are single trials; large markers are condition means.*

With three neurons the space has three axes, and the picture still works (the notebook draws it in 3D). With a hundred neurons it can no longer be drawn, but the mathematics is exactly the same: one axis per neuron, one point per state.

1.4. Writing vectors down, and storing them

▶ In the teaching notebook: Part 3

Columns and rows

Mathematics usually writes a vector as a **column**, and its **transpose**, written with a superscript T, as a row:

$$\mathbf{v} = \begin{pmatrix} v_1 \\ v_2 \\ v_3 \end{pmatrix}, \qquad \mathbf{v}^{\mathsf{T}} = \begin{pmatrix} v_1 & v_2 & v_3 \end{pmatrix}.$$

Writing (v_1, v_2, v_3) inside a sentence is shorthand for the column. For now the orientation is only a convention. From Lesson 3, when vectors meet matrices, it starts to matter.

Shapes in NumPy

NumPy stores the same three numbers in three different ways, distinguished by their **shape**:

Table 1.3 *Three NumPy shapes for the same three numbers.*

Shape	Created by	What it is
`(3,)`	`np.array([1., 2., 3.])`	a flat, one-dimensional array with no orientation
`(3, 1)`	`v.reshape(3, 1)`	a column: three rows, one column
`(1, 3)`	`v.reshape(1, 3)`	a row: one row, three columns

The flat shape `(n,)` is the usual way to store a vector in NumPy. Two attributes describe any array: `.shape`, the size along each axis, and `.ndim`, the number of axes.

PYTHON

```
r = np.array([12.5, 45.3, 8.7, 33.1, 71.4])   # five neurons, spikes/s
print(r.shape, r.ndim, r.size)                 # -> (5,) 1 5
print(r[0], r[-1], r[1:3])                     # -> 12.5 71.4 [45.3  8.7]
```

Python counts from zero, so neuron N1 is `r[0]`. A slice `r[1:3]` includes the start index and stops *before* the end index, giving N2 and N3.

Broadcasting

When NumPy combines two arrays of different shapes, it applies a rule called **broadcasting**: it lines the shapes up from the right, and any axis of size 1 is stretched to match the other array. This is often convenient and occasionally disastrous.

PYTHON

```
v_flat = np.array([1.0, 2.0, 3.0])   # shape (3,)
v_col  = v_flat.reshape(3, 1)        # shape (3, 1)
print((v_flat + v_col).shape)        # -> (3, 3)   not (3,)!
```

The flat array is treated as a row of shape `(1, 3)`; the column has shape `(3, 1)`. Both size-1 axes stretch, and the result is a 3×3 table holding every pairwise sum. No error is raised.

> **COMMON PITFALL**
>
> Before combining two arrays, print both `.shape`s. If they differ, reshape one to match, for example with `v_col.reshape(-1)`, which flattens back to `(3,)`. Most shape bugs in neural-data code are broadcasting doing exactly what it was told.

1.5. Adding, subtracting and scaling

▶ In the teaching notebook: Parts 4 and 5

DEFINITION – VECTOR ADDITION AND SCALAR MULTIPLICATION

For $\mathbf{u}, \mathbf{v} \in \mathbb{R}^n$ and a scalar c, act coordinate by coordinate:

$$\mathbf{u} + \mathbf{v} = (u_1 + v_1,\ \ldots,\ u_n + v_n), \qquad \mathbf{u} - \mathbf{v} = (u_1 - v_1,\ \ldots,\ u_n - v_n),$$
$$c\,\mathbf{v} = (cv_1,\ \ldots,\ cv_n).$$

Only vectors of the same dimension can be added or subtracted.

WORKED EXAMPLE – BASELINE AND STIMULUS

Three neurons, mean rates under two conditions: Baseline $\mathbf{r}_A = (5,\ 8,\ 6)$ and Stimulus A $\mathbf{r}_B = (55,\ 12,\ 9)$ spikes/s.

$\mathbf{r}_B - \mathbf{r}_A = (55-5,\ 12-8,\ 9-6) = (50,\ 4,\ 3)$	the change in state
$1.5\,\mathbf{r}_A = (7.5,\ 12,\ 9)$	same pattern, 50% stronger
$\mathbf{r}_A + \mathbf{r}_B = (60,\ 20,\ 15)$	a modelled combination

What the operations look like

Addition places arrows tip to tail: walk along $\mathbf{p}$, then along $\mathbf{q}$, and you arrive at $\mathbf{p} + \mathbf{q}$. Doing it in the other order arrives at the same place, so the two routes form a parallelogram (Figure 1.3).

Subtraction gives the arrow *from* one point *to* another. $\mathbf{r}_B - \mathbf{r}_A$ is the arrow that starts at state A and ends at state B, because $\mathbf{r}_A + (\mathbf{r}_B - \mathbf{r}_A) = \mathbf{r}_B$ (Figure 1.4). This is the state-change vector.

Scalar multiplication stretches or shrinks an arrow without turning it (if $c > 0$), or flips it to point the opposite way (if $c < 0$). For a population response, $c\,\mathbf{r}$ is the same *pattern* across neurons at a different overall *strength*.

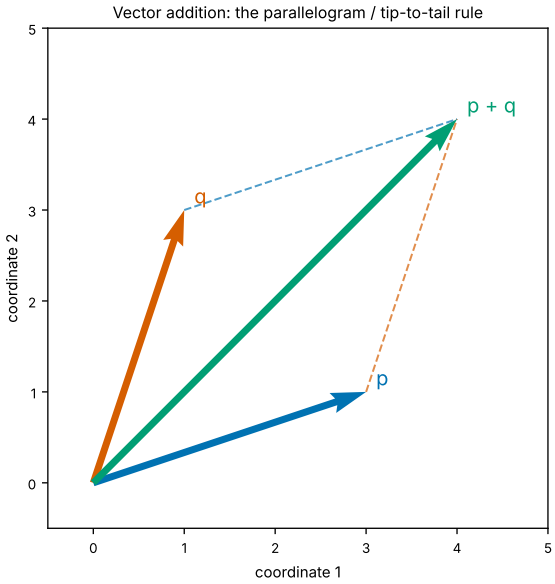


Figure 1.3 *Addition.*

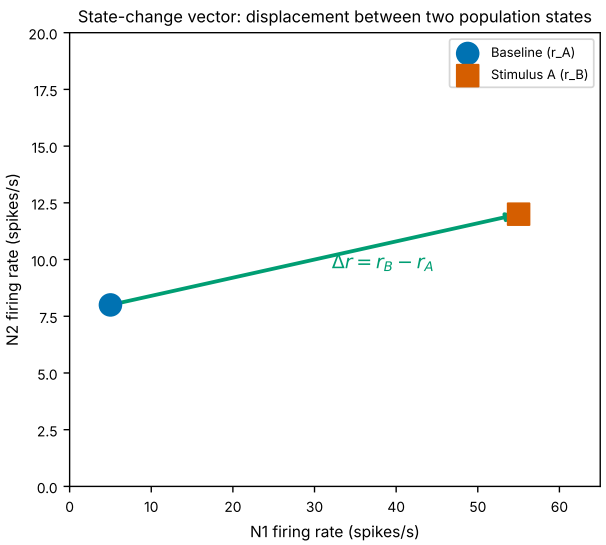


Figure 1.4 *Subtraction.*

The rules of vector arithmetic

Because each operation works coordinate by coordinate, vectors inherit the familiar rules of ordinary arithmetic.

Table 1.4 *Rules of vector arithmetic.* $\mathbf{u}, \mathbf{v}, \mathbf{w} \in \mathbb{R}^n$; a, c *are scalars.*

Rule	Statement
Order of addition does not matter	$\mathbf{u} + \mathbf{v} = \mathbf{v} + \mathbf{u}$
Grouping does not matter	$(\mathbf{u} + \mathbf{v}) + \mathbf{w} = \mathbf{u} + (\mathbf{v} + \mathbf{w})$
Zero vector	$\mathbf{v} + \mathbf{0} = \mathbf{v}$
Negatives	$\mathbf{v} + (-\mathbf{v}) = \mathbf{0}$, where $-\mathbf{v} = (-1)\mathbf{v}$
Scaling a sum	$c(\mathbf{u} + \mathbf{v}) = c\mathbf{u} + c\mathbf{v}$
Summing scales	$(a + c)\mathbf{v} = a\mathbf{v} + c\mathbf{v}$
Scaling twice	$a(c\mathbf{v}) = (ac)\mathbf{v}$, and $1\mathbf{v} = \mathbf{v}$

WHY IT IS TRUE – ALL SEVEN RULES

Each rule is true for ordinary numbers, and vector operations apply ordinary arithmetic to each coordinate separately. For example, coordinate i of $c(\mathbf{u} + \mathbf{v})$ is $c(u_i + v_i) = cu_i + cv_i$, which is coordinate i of $c\mathbf{u} + c\mathbf{v}$. Since this holds for every i, the vectors are equal. The other six rules are checked the same way.

Any collection of objects that can be added and scaled while obeying these rules is called a **vector space**. The name matters because so many things qualify: lists of firing rates, sound waveforms, images, even functions (Week 3). Everything proved from these rules holds for all of them.

COMMON PITFALL

Arithmetic is always *possible*; it is not always *meaningful*. Adding two coordinates measured in different units, or adding two population responses and calling the sum "the response to both stimuli together", is a modelling claim. Real neurons often combine inputs nonlinearly. Treat $\mathbf{r}_A + \mathbf{r}_B$ as a hypothesis to test, not a fact.

1.6. Length and distance

▶ In the teaching notebook: Part 6

From Pythagoras to the norm

How long is the arrow $\mathbf{v} = (3, 4)$? Its two coordinates are the two shorter sides of a right-angled triangle, and the arrow is the hypotenuse, so Pythagoras gives $\sqrt{3^2 + 4^2} = 5$ (Figure 1.5, left).

DEFINITION – EUCLIDEAN NORM

The **Euclidean norm**, or length, of $\mathbf{v} \in \mathbb{R}^n$ is

$$\|\mathbf{v}\| = \sqrt{v_1^2 + v_2^2 + \cdots + v_n^2} = \sqrt{\textstyle\sum_{i=1}^{n} v_i^2}.$$

It is sometimes written $\|\mathbf{v}\|_2$ to distinguish it from other norms.

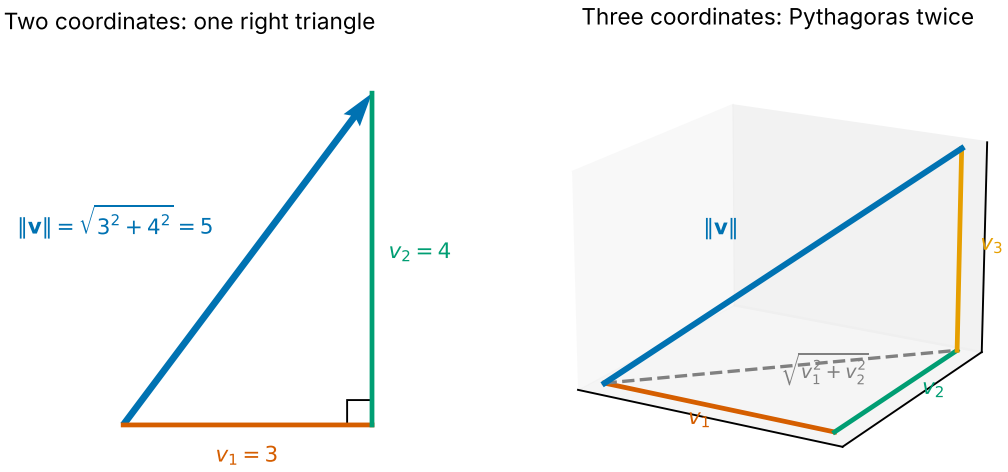


Figure 1.5 *The length of a vector is Pythagoras' theorem, applied once in two dimensions and twice in three.*

WHY IT IS TRUE – THE FORMULA IN ANY NUMBER OF DIMENSIONS

In three dimensions, apply Pythagoras twice (Figure 1.5, right). The diagonal of the floor of the box has squared length $v_1^2 + v_2^2$. That diagonal and the vertical side v_3 are the two shorter sides of a second right-angled triangle, whose hypotenuse is the vector. So $\|\mathbf{v}\|^2 = (v_1^2 + v_2^2) + v_3^2$.
Each new coordinate adds one more right-angled step, so in n dimensions $\|\mathbf{v}\|^2 = v_1^2 + \cdots + v_n^2$. When we can no longer draw the space, we *define* length by this formula, so that it agrees with geometry wherever geometry can be drawn.

Properties of the norm

Three properties make the norm behave like a length.

1. **Never negative, and zero only for the zero vector.** A sum of squares is ≥ 0, and is 0 only if every $v_i = 0$.
2. **Scaling scales the length.** $\|c\mathbf{v}\| = |c|\,\|\mathbf{v}\|$. Doubling a response doubles its length; reversing it does not change its length.
3. **The triangle inequality.** $\|\mathbf{u} + \mathbf{v}\| \leq \|\mathbf{u}\| + \|\mathbf{v}\|$. Going directly is never longer than going via a detour (Figure 1.6).

WHY IT IS TRUE – $\|c\mathbf{v}\| = |c|\,\|\mathbf{v}\|$

$\|c\mathbf{v}\| = \sqrt{\sum_i (cv_i)^2} = \sqrt{c^2 \sum_i v_i^2} = \sqrt{c^2}\,\sqrt{\sum_i v_i^2} = |c|\,\|\mathbf{v}\|$. Note $\sqrt{c^2} = |c|$, not c: length is never negative.

The triangle inequality is proved in Lesson 2, once the dot product is available (it follows from the Cauchy–Schwarz inequality proved there).

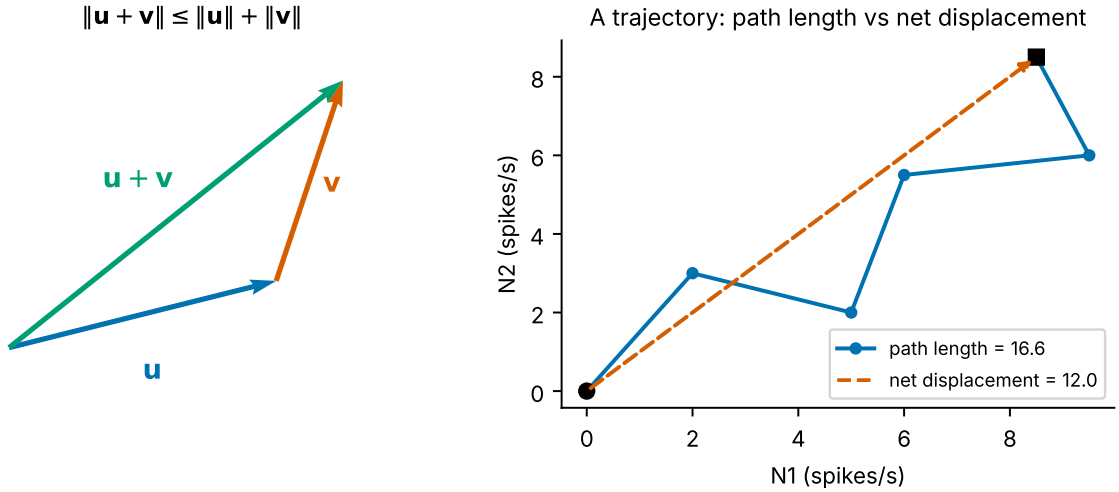


Figure 1.6 *Left: the triangle inequality. Right: its consequence for a trajectory through state space, used in Section 1.9. The total path length can never be shorter than the straight-line displacement from start to end.*

Distance

DEFINITION – EUCLIDEAN DISTANCE

The distance between two vectors is the length of their difference,

$$d(\mathbf{u}, \mathbf{v}) = \|\mathbf{u} - \mathbf{v}\| = \sqrt{\textstyle\sum_{i=1}^{n} (u_i - v_i)^2}.$$

That is exactly the length of the state-change arrow from Section 1.5.

For Baseline and Stimulus A, $d = \|(50,\ 4,\ 3)\| = \sqrt{2500 + 16 + 9} = \sqrt{2525} = 50.2$ spikes/s. Distance inherits three sensible properties from the norm:

- $d(\mathbf{u}, \mathbf{v}) \geq 0$ with equality only when $\mathbf{u} = \mathbf{v}$;
- $d(\mathbf{u}, \mathbf{v}) = d(\mathbf{v}, \mathbf{u})$;
- $d(\mathbf{u}, \mathbf{w}) \leq d(\mathbf{u}, \mathbf{v}) + d(\mathbf{v}, \mathbf{w})$.

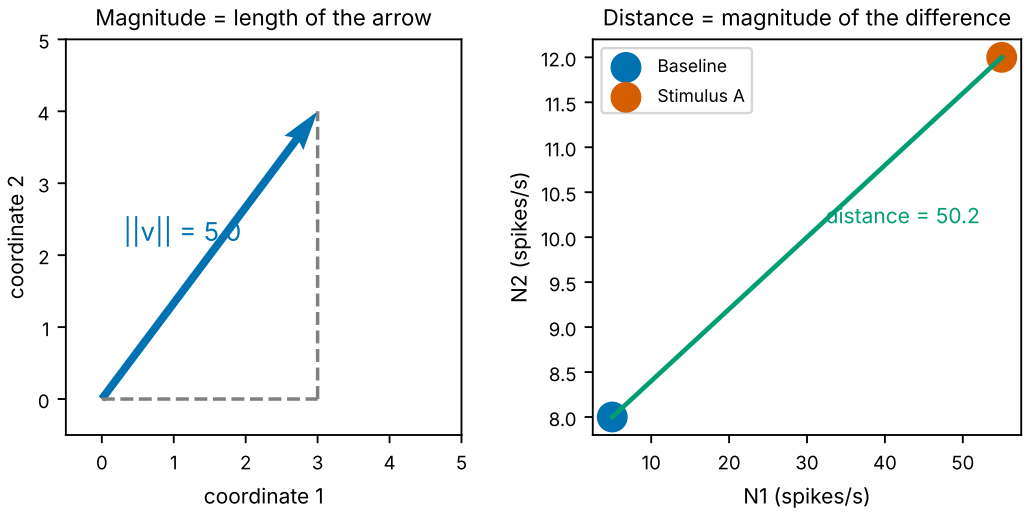


Figure 1.7 *Length is the size of one arrow (left); distance is the length of the difference between two states (right, first two neurons only).*

PYTHON

```
d = np.linalg.norm(condition_means['Baseline'] - condition_means['Stimulus A'])
print(round(d, 1))     # -> 50.2
```

Repeating this for every pair of the four conditions gives Table 1.5.

Table 1.5 *Euclidean distances between the four condition means (spikes/s, all three neurons).*

	Baseline	Stimulus A	Stimulus B	Motor
Baseline	0.0	50.2	65.2	63.4
Stimulus A	50.2	0.0	75.0	61.1
Stimulus B	65.2	75.0	0.0	36.5
Motor	63.4	61.1	36.5	0.0

A table of all pairwise distances is symmetric with zeros down the diagonal, as the properties require. Stimulus B and Motor are the closest pair of distinct conditions (36.5); Stimulus A and Stimulus B the furthest apart (75.0).

Other ways to measure length

The Euclidean norm is the natural length of an arrow, but not the only useful one.

$$\|\mathbf{v}\|_1 = \sum_i |v_i| \quad \text{(Manhattan)}, \qquad \|\mathbf{v}\|_\infty = \max_i |v_i| \quad \text{(largest coordinate)}.$$

The L_1 norm is the distance a taxi drives along a street grid; the L_∞ norm is the biggest change in any one neuron. Figure 1.8 shows the set of all vectors of length 1 under each norm. In machine learning, the L_1 and L_2 norms of a weight vector reappear as *regularisation* penalties (Week 4).

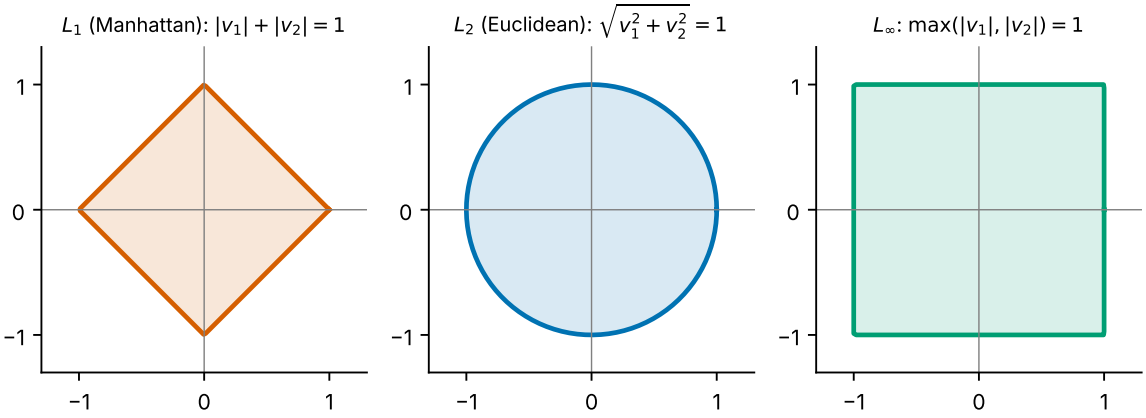


Figure 1.8 *All two-dimensional vectors of length 1, under three different norms.*

When distance misleads

Euclidean distance quietly assumes every coordinate is measured on a comparable scale. Two houses differ by 114 square feet, 1 bedroom and 35 years. Squaring and adding, the size difference supplies 91% of the squared distance and the bedrooms almost nothing, simply because square feet are big numbers (Figure 1.9). The same happens with neurons: one that ranges from 0 to 200 spikes/s will dominate one that ranges from 0 to 5, regardless of which carries the information.

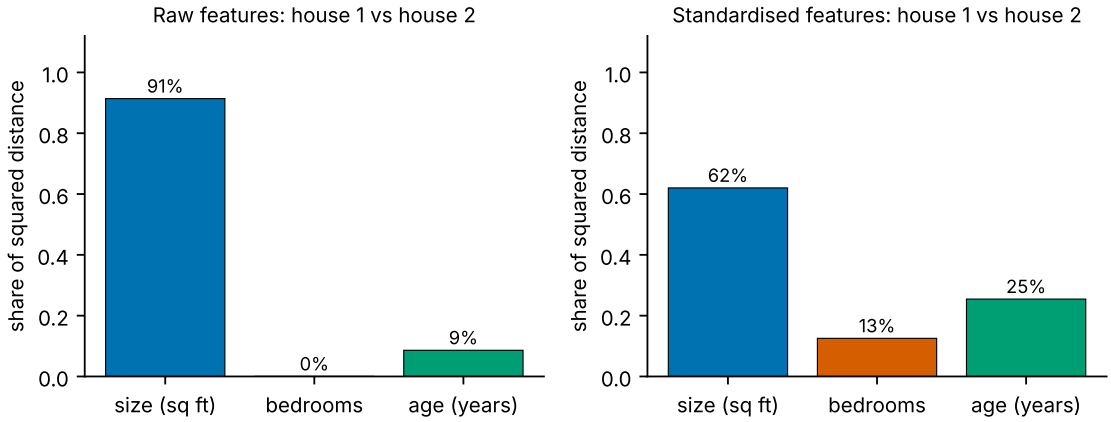


Figure 1.9 *How much each feature contributes to the squared distance between two houses, before and after standardising each feature.*

A common fix is **standardisation**: for each feature, subtract its mean across the dataset and divide by its standard deviation, $z = (x - \text{mean})/\text{sd}$. Each feature then has no units and a comparable spread. Week 2 returns to this choice in detail.

> **KEY POINT**
>
> Euclidean distance is a modelling choice, not an automatic measure of biological or practical similarity. Check the units and ranges of the coordinates before trusting it.

1.7. Unit vectors and normalisation

▶ In the teaching notebook: Part 7

Sometimes the *pattern* of a response matters more than its strength. The same stimulus at low and high contrast may drive the same neurons in the same proportions, only more strongly. To compare patterns, remove the strength.

> **DEFINITION – UNIT VECTOR AND NORMALISATION**
>
> A **unit vector** has length 1. For any nonzero $\mathbf{v}$, the unit vector pointing the same way is
>
> $$\hat{\mathbf{v}} = \frac{\mathbf{v}}{\|\mathbf{v}\|}.$$
>
> Computing $\hat{\mathbf{v}}$ is called **normalisation**. The zero vector has no direction and cannot be normalised.

> **WHY IT IS TRUE – $\hat{\mathbf{v}}$ HAS LENGTH 1 AND POINTS THE SAME WAY**
>
> $\hat{\mathbf{v}}$ is $\mathbf{v}$ multiplied by the positive scalar $c = 1/\|\mathbf{v}\|$, so it points the same way. By the scaling property, $\|\hat{\mathbf{v}}\| = |c|\,\|\mathbf{v}\| = \|\mathbf{v}\|/\|\mathbf{v}\| = 1$.

Rearranging the definition shows what normalisation does:

$$\mathbf{v} = \underbrace{\|\mathbf{v}\|}_{\text{strength}} \times \underbrace{\hat{\mathbf{v}}}_{\text{pattern}}.$$

Every nonzero vector splits into a length and a direction. Normalisation keeps the direction and discards the length. For $\mathbf{v} = (3,\ 4)$: $\|\mathbf{v}\| = 5$ and $\hat{\mathbf{v}} = (0.6,\ 0.8)$.

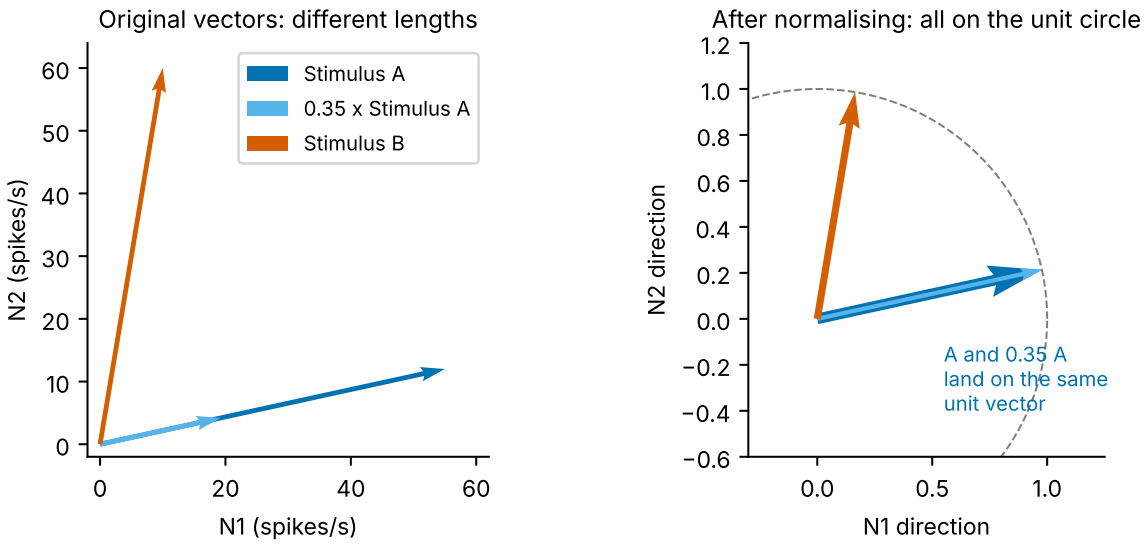


Figure 1.10 *Left: three responses in two-neuron space. Right: after normalising, each lies on the unit circle. Stimulus A and a weaker copy of it land on exactly the same unit vector.*

In code, normalisation needs a guard: dividing by a length of zero is undefined, and NumPy would silently return `nan`. Checking against a small tolerance `eps` also catches vectors that are zero up to rounding.

PYTHON

```
def normalise(v, eps=1e-12):
    v = np.asarray(v, dtype=float)
    norm = np.linalg.norm(v)
    if norm < eps:
        raise ValueError("The zero vector has no defined direction.")
    return v / norm

print(normalise([3.0, 4.0]))     # -> [0.6 0.8]
```

IN PRACTICE

What normalisation removes: overall response strength. **What it keeps:** the relative pattern across neurons. Whether that is the right trade depends on the question. If arousal or contrast changes overall firing and you care about *which* neurons respond, normalise. If response strength itself is the signal, do not. Lesson 2 builds on this idea to measure how similar two patterns are (cosine similarity).

1.8. Linear combinations, span, basis and dimension

▶ In the teaching notebook: Part 8

Linear combinations

DEFINITION – LINEAR COMBINATION

A **linear combination** of vectors $\mathbf{v}_1, \ldots, \mathbf{v}_k$ is any weighted sum

$$\alpha_1 \mathbf{v}_1 + \alpha_2 \mathbf{v}_2 + \cdots + \alpha_k \mathbf{v}_k,$$

where the weights $\alpha_1, \ldots, \alpha_k$ are scalars.

Suppose the population often responds in two coordinated patterns: a "sensory" pattern $\mathbf{u}_{\text{stim}} = (1,\ 1,\ 0)$ that engages N1 and N2, and a "motor" pattern $\mathbf{u}_{\text{motor}} = (0,\ 1,\ 1)$ that engages N2 and N3. On a trial that engages them with strengths 20 and 15,

$$20\,\mathbf{u}_{\text{stim}} + 15\,\mathbf{u}_{\text{motor}} = (20,\ 20,\ 0) + (0,\ 15,\ 15) = (20,\ 35,\ 15).$$

The standard basis and coordinates

The simplest building blocks are the **standard basis** vectors, each with a single 1:

$$\mathbf{e}_1 = (1, 0, \ldots, 0), \quad \mathbf{e}_2 = (0, 1, 0, \ldots, 0), \quad \ldots, \quad \mathbf{e}_n = (0, \ldots, 0, 1).$$

Every vector is a linear combination of them, and the weights are just its coordinates:

$$\mathbf{v} = v_1 \mathbf{e}_1 + v_2 \mathbf{e}_2 + \cdots + v_n \mathbf{e}_n.$$

For $\mathbf{v} = (4,\ -2)$: four steps along $\mathbf{e}_1$, then -2 along $\mathbf{e}_2$ (Figure 1.11). In neural terms, $\mathbf{e}_i$ is "neuron i alone", and a population vector is a weighted sum of single-neuron directions.

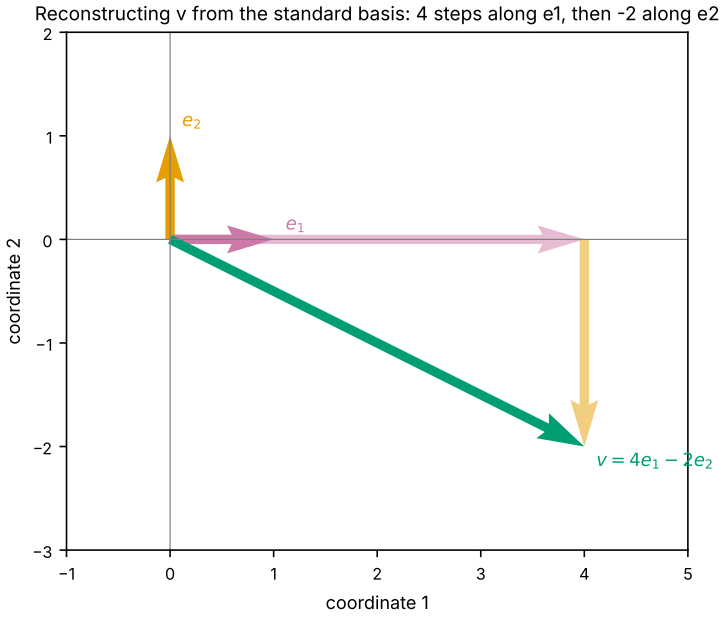


Figure 1.11 *Rebuilding* $\mathbf{v} = (4, -2)$ *from the standard basis.*

Span and linear independence

DEFINITION – SPAN

The **span** of a set of vectors is the set of all their linear combinations: every point you can reach by choosing the weights.

One nonzero vector spans a line through the origin. Two vectors span a plane, *unless* one is a multiple of the other, in which case they still only span a line (Figure 1.12). The second vector added no new direction.

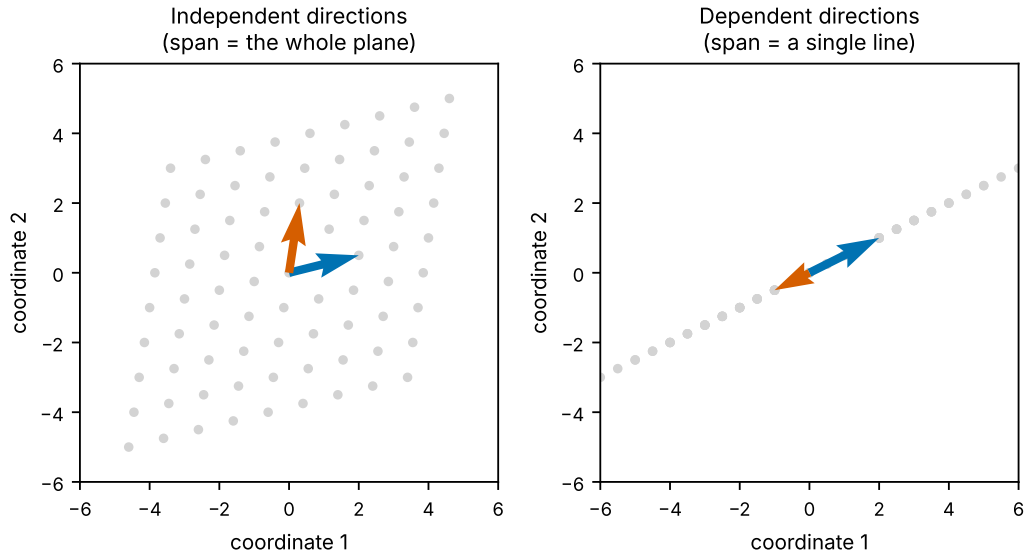


Figure 1.12 *Grey dots are linear combinations with weights between* -2 *and* 2*. Independent directions reach the whole plane; dependent ones stay on a line.*

DEFINITION – LINEAR INDEPENDENCE

Vectors $\mathbf{v}_1, \ldots, \mathbf{v}_k$ are **linearly independent** if the only way to combine them into the zero vector is with every weight zero:

$$\alpha_1 \mathbf{v}_1 + \cdots + \alpha_k \mathbf{v}_k = \mathbf{0} \quad \Longrightarrow \quad \alpha_1 = \cdots = \alpha_k = 0.$$

Otherwise they are **linearly dependent**: at least one of them is a combination of the others, and adds nothing to the span.

For two vectors the test is simple: they are dependent exactly when one is a multiple of the other. $(2,\ 3)$ and $(4,\ 6)$ are dependent, since $(4,6) = 2(2,3)$. $(1,\ 0)$ and $(0,\ 1)$ are independent.

Basis, coordinates and dimension

DEFINITION – BASIS AND DIMENSION

A **basis** of a space is a set of linearly independent vectors whose span is the whole space. Every vector in the space is then a combination of the basis vectors in exactly one way, and those unique weights are its **coordinates** in that basis. Every basis of a given space has the same number of vectors, and that number is the space's **dimension**.

The standard basis $\mathbf{e}_1, \ldots, \mathbf{e}_n$ is one basis of $\mathbb{R}^n$, which is why $\mathbb{R}^n$ has dimension n. But it is not the only one, and other bases are often more meaningful.

WHY IT IS TRUE – COORDINATES IN A BASIS ARE UNIQUE

Suppose $\mathbf{v} = \sum_i \alpha_i \mathbf{b}_i = \sum_i \beta_i \mathbf{b}_i$ for the same basis vectors $\mathbf{b}_i$. Subtracting, $\sum_i (\alpha_i - \beta_i)\mathbf{b}_i = \mathbf{0}$. The basis vectors are independent, so every $\alpha_i - \beta_i = 0$. The two sets of weights were the same. □

WORKED EXAMPLE – IS A STATE IN THE SPAN OF TWO PATTERNS?

Every combination of $\mathbf{u}_{\text{stim}}$ and $\mathbf{u}_{\text{motor}}$ has the form

$$a(1,\ 1,\ 0) + b(0,\ 1,\ 1) = (a,\ a+b,\ b).$$

State $(20,\ 35,\ 15)$**.** Matching the first and last coordinates forces $a = 20$, $b = 15$. Check the middle one: $a + b = 35$. It matches, so the state is in the span, with coordinates $(20,\ 15)$ in the pattern basis.
State $(30,\ 10,\ 0)$**.** Now $a = 30$, $b = 0$, but then $a + b = 30 \neq 10$. No weights work: this state is *not* in the span.

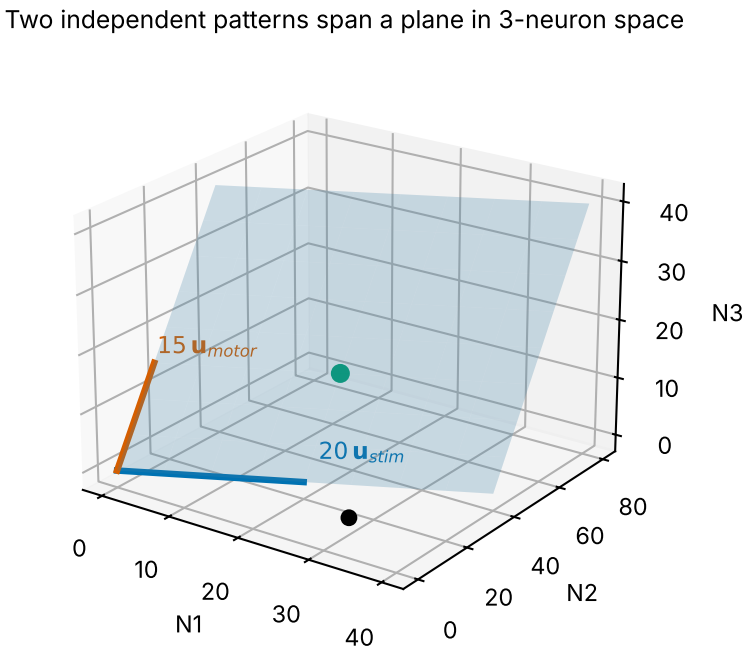


Figure 1.13 *The two patterns span a plane inside three-neuron space. The green point* $(20,\ 35,\ 15)$ *is built from them and lies on the plane; the black point* $(30,\ 10,\ 0)$ *needs a third pattern and lies off it.*

KEY POINT

Recordings of hundreds of neurons often turn out to be well described by a handful of coordinated patterns: the activity stays close to a low-dimensional subspace of the full neural space, sometimes called a **neural manifold**. Finding those patterns from data is what principal component analysis does (Week 2, Lesson 7).

1.9. From time series to trajectories

▶ In the teaching notebook: Parts 9 and 10

Real recordings follow activity through time. At each time bin t the population has a state vector $\mathbf{r}(t)$, and the ordered sequence of states $\mathbf{r}(0), \mathbf{r}(1), \ldots$ is a **trajectory** through state space.

Stored in NumPy, a recording of T time bins from n neurons is an array of shape `(T, n)`: one row per time bin, one column per neuron. Row t is the state vector $\mathbf{r}(t)$; column i is neuron i's firing-rate trace. The same numbers can be drawn as n traces against time, or as one path through n-dimensional space (Figures 1.14 and 1.15).

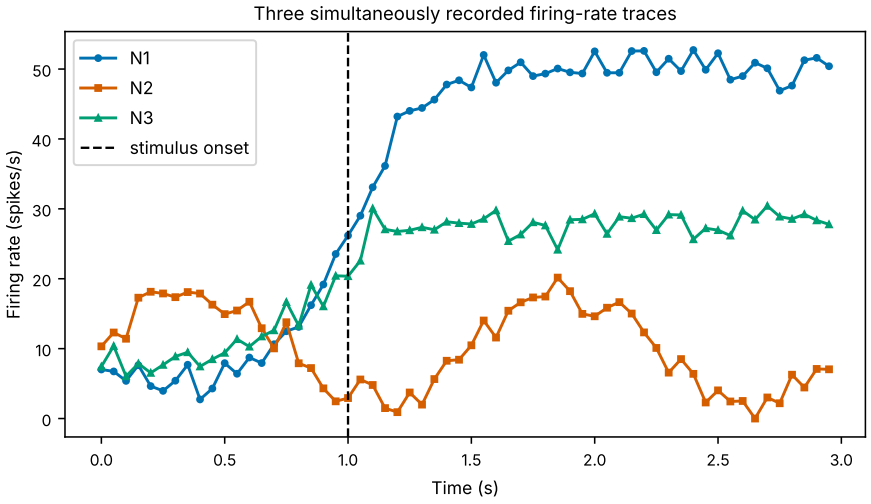


Figure 1.14 *Three simultaneously recorded firing-rate traces, with a stimulus at 1 s. Each column of the recording array is one trace.*

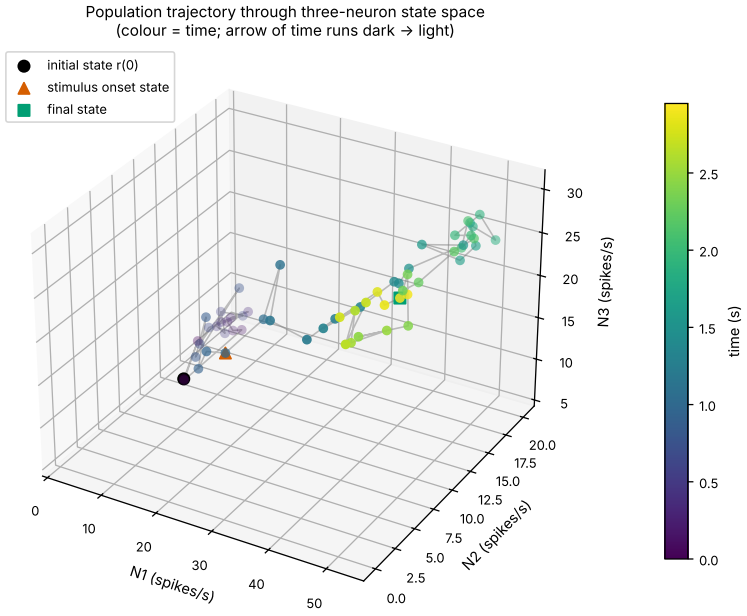


Figure 1.15 *The same recording as one trajectory through three-neuron state space. Each point is one row of the array; colour shows time.*

Everything from this lesson applies to the trajectory.

- The **step** from one bin to the next is the difference $\Delta\mathbf{r}(t) = \mathbf{r}(t+1) - \mathbf{r}(t)$.
- Its length divided by the bin width Δt, $\|\Delta\mathbf{r}(t)\| / \Delta t$, is the **speed** of the population through state space. Where consecutive points are close, the state changed slowly; where they spread apart, it changed quickly.
- The **path length** adds up all the step lengths. The **net displacement** is the distance from the first state to the last. By the triangle inequality the path is never shorter than the displacement (Figure 1.6, right). A large ratio between them means the population wandered rather than moving directly.

Comparing two conditions then means comparing two trajectories: for example, the distance between their states at matching times. In the notebook's mini-lab the two conditions start 2.1 spikes/s apart and end 24.0 spikes/s apart, consistent with the stimuli driving increasingly distinct patterns over time.

> **IN PRACTICE**
>
> Reaching movements in monkeys provide a classic example. Motor cortex activity during a reach traces out smooth, rotation-like trajectories that are remarkably similar across reach directions (Churchland et al., 2012). Analyses like that one start from exactly the objects in this section: a `(T, n)` array, its rows as states, and distances and directions between them.

Lesson summary

- A vector is an ordered list of n numbers treated as one object; its meaning lives in what each coordinate refers to and its units.
- A vector can be read as a list, an arrow (a change) or a point (a state); neural data mostly uses points in state space.
- NumPy stores vectors as shape `(n,)`; columns `(n, 1)` and rows `(1, n)` broadcast differently.
- Addition, subtraction and scaling work coordinate by coordinate; $\mathbf{r}_B - \mathbf{r}_A$ is the change from state A to state B, and $c\,\mathbf{r}$ is the same pattern at a different strength.
- The norm $\|\mathbf{v}\| = \sqrt{\sum_i v_i^2}$ is Pythagoras in n dimensions; $\|c\mathbf{v}\| = |c|\|\mathbf{v}\|$; the triangle inequality holds.
- Distance is the norm of the difference; it is a modelling choice that is distorted by mismatched scales.
- Normalisation $\hat{\mathbf{v}} = \mathbf{v}/\|\mathbf{v}\|$ splits a vector into strength $\times$ pattern and keeps only the pattern.
- Linear combinations, span, independence, basis and dimension describe which states a set of patterns can produce; population activity often stays near a low-dimensional subspace.
- A recording through time is a `(T, n)` array whose rows trace a trajectory; steps, speed, path length and displacement are all differences and norms.

Next: Lesson 2, Dot products and projection. Lesson 1 measured one vector at a time. Lesson 2 asks how two vectors relate: how aligned their directions are, how much of one lies along the other, and how a neuron combines its inputs.

Exercises

These are pen-and-paper exercises that complement the coding exercises in the exercise notebook. Answers will be released the following week.

1. Give the dimension of each vector, and say what one coordinate represents: (a) one time point from a 32-channel EEG recording; (b) one 20×20 pixel image, flattened; (c) the mean firing rates of 150 neurons under one condition.
2. For $\mathbf{u} = (3,\ -1,\ 2)$ and $\mathbf{v} = (1,\ 4,\ -2)$, compute $\mathbf{u} + \mathbf{v}$, $\mathbf{u} - \mathbf{v}$ and $3\mathbf{u} - 2\mathbf{v}$.
3. Compute $\|(2,\ -3,\ 6)\|$, and the distance between $(1,\ 2,\ 3)$ and $(4,\ 6,\ 3)$.
4. Prove that $\|c\mathbf{v}\| = |c|\|\mathbf{v}\|$, and use it to show that $\|\hat{\mathbf{v}}\| = 1$ for every nonzero $\mathbf{v}$.
5. Normalise $(0,\ 5,\ 12)$ and $(-2,\ -2)$. Then explain why $(1,\ 2)$ and $(10,\ 20)$ have the same unit vector, and what that means if they are two population responses.
6. A vector holds a neuron's firing rate (spikes/s) and its membrane potential (mV). Which of these make sense: adding two such vectors; scaling one by 2; computing the Euclidean distance between two of them? Explain.
7. What are the NumPy shapes of `a + b` when `a` has shape `(3,)` and `b` has shape `(1, 3)`? And when `a` has shape `(3, 1)` and `b` has shape `(1, 3)`? Explain using the broadcasting rule.
8. Express $(7,\ 1)$ as a combination of $\mathbf{b}_1 = (1,\ 1)$ and $\mathbf{b}_2 = (1,\ -1)$. What are its coordinates in this basis?
9. Is $(3,\ 8,\ 5)$ in the span of $\mathbf{u}_{\text{stim}} = (1, 1, 0)$ and $\mathbf{u}_{\text{motor}} = (0, 1, 1)$? Is $(2,\ 1,\ 2)$? Show your working.
10. Are $(1,\ 2,\ 0)$, $(0,\ 1,\ 1)$ and $(1,\ 3,\ 1)$ linearly independent? What is the dimension of their span?
11. A two-neuron state moves through $\mathbf{r}(0) = (10,\ 10)$, $\mathbf{r}(1) = (13,\ 14)$, $\mathbf{r}(2) = (13,\ 20)$ spikes/s, with bins 0.5 s apart. Compute the two step lengths, the path length, the net displacement, and the average speed through state space.
12. Two patients are described by (age in years, cholesterol in mg/dL): $(30,\ 200)$ and $(60,\ 210)$. Compute their Euclidean distance. Which feature dominates it, and would that change if cholesterol were recorded in g/dL instead?
13. (*Challenge.*) Show that for every vector, $\|\mathbf{v}\|_\infty \le \|\mathbf{v}\|_2 \le \|\mathbf{v}\|_1$. Check it on $\mathbf{v} = (3,\ -4,\ 12)$.

Further reading

- G. Strang, *Introduction to Linear Algebra*, 6th ed., Wellesley–Cambridge Press, 2023. Chapter 1 introduces vectors, linear combinations and lengths.
- M. M. Churchland, J. P. Cunningham, M. T. Kaufman, J. D. Foster, P. Nuyujukian, S. I. Ryu and K. V. Shenoy, "Neural population dynamics during reaching", *Nature* 487:51–56, 2012.
- S. Saxena and J. P. Cunningham, "Towards the neural population doctrine", *Current Opinion in Neurobiology* 55:103–111, 2019.
- J. A. Gallego, M. G. Perich, L. E. Miller and S. A. Solla, "Neural manifolds for the control of movement", *Neuron* 94(5):978–984, 2017.